

Geranium: Multimodal Retrieval of Genomics Data Visualizations

Huyen N. Nguyen , Sehi L'Yi , Thomas C. Smits , Shanghua Gao , Marinka Zitnik , Nils Gehlenborg 

Abstract—Effective visualization is essential for interpreting genomics data, yet researchers often face challenges in finding relevant, reusable examples. Existing tools offer limited support for searching the vast landscape of genomics visualizations, making the process of authoring new visualizations time-consuming and inefficient. To address this gap, we introduce Geranium, a data visualization retrieval system for searching and authoring genomics visualizations. Geranium supports multimodal retrieval, enabling users to query with images, text, or grammar-based specifications. Retrieved examples serve as scaffolds for authoring, providing templates that researchers can adapt with their own data, thereby streamlining the mechanics of visualization construction. Geranium integrates three embedding methods to combine specialized and general knowledge: grammar-based embeddings tailored to genomics visualizations, multimodal embeddings from a biomedical vision-language foundation model, and text embeddings from a fine-tuned large language model. For each visualization, we construct a multimodal representation that includes a Gosling specification, a pixel-based rendering, and natural language descriptions. We evaluate embedding strategies to maximize top- k retrieval accuracy and conduct user studies with domain collaborators to gather feedback on usability. Our collection comprises 3,200 visualizations across 50 categories, ranging from single-view to coordinated multi-view designs and supporting applications from single-cell epigenomics to structural variation analysis.

Index Terms—Visualization retrieval, multimodal retrieval, multimodal representation, visualization authoring, genomics data visualization.

I. INTRODUCTION

Interactive visualizations have become increasingly essential in genomics research, enabling scientists and clinicians to discover patterns and analyze complex biological data. As the scale and heterogeneity of genomics datasets grow, researchers have developed a variety of visualization techniques to navigate such complexity. However, this progress has resulted in a vast yet fragmented landscape of tools and examples [1], [2], making it difficult for scientists to find visualizations that meet their needs. This challenge necessitates new systems that can efficiently retrieve relevant genomics visualizations, enabling researchers to explore and build on existing examples.

Our work is motivated by a recent interview study with 20 genomics experts [3], which highlighted the role of visualization authoring in genomics workflows. In this context, visualization authoring refers to the process of transforming data into visual representations. A key finding is that visualization authoring often begins with adapting existing examples:

researchers typically search online for visualization templates, copy the code that approximates their intended result, and modify it to suit their data and tasks. This practice in genomics echoes findings on general visualization workflows by Bako et al. [4]. However, modifiable examples for genomics visualizations (e.g., for BAM and GFF file formats) are much more limited than those for general visualizations (e.g., Vega-Lite [5] for CSV data). More importantly, besides the lack of adaptable templates, there are also fewer solutions and tools that facilitate this type of workflow. As a result, genomics researchers must often build visualizations from scratch or devote extra time and effort to searching, making this process time-consuming and inefficient.

This bottleneck in the workflow raises two important questions: first, where can we obtain a collection of genomics visualization examples, and second, how can we effectively query that collection to find relevant examples? Previous work has addressed the problem of retrieving general visualizations [6]–[10]; however, these approaches do not naturally extend to genomics visualizations. Genomics-specific visual encodings such as sequence logos and ideograms, and data types such as expression levels and phenotype information, are largely absent from the training corpora of general visualization embedding models, limiting the applicability of existing retrieval approaches. In addition, the multi-scale nature of genomics data complicates retrieval, as many genomics visualizations rely on semantic zooming, causing visual representations to appear significantly different across scales, ranging from whole chromosomes to individual nucleotides. For example, a peak pattern in a genomic signal track may compress, stretch, or disappear depending on the zoom level, making visual matching difficult.

In this work, we focus on genomics visualization retrieval, a key challenge in the visualization authoring process. Our goal is to enable the efficient retrieval of interactive genomics visualizations that serve as adaptable templates. We introduce **Geranium** (Genomics Visualization Retrieval for Authoring with Multimodality), a data visualization retrieval system that supports multimodal search specialized for the genomics domain. This search capacity allows users to query with images, text keywords, or partial specifications used to build the visualization. The retrieved examples serve as *scaffolds* in the authoring process: templates that researchers can fill in or modify with their own data and design preferences, allowing them to focus on data analysis rather than the mechanics of visualization construction.

A key component of our approach is the construction of multimodal representations for each visualization. Each

The authors are with Department of Biomedical Informatics, Harvard Medical School, Boston, MA 02115, United States. E-mail: {huyen_nguyen | sehi_lyi | tsmits | shanghua_gao | marinka | nils}@hms.harvard.edu.

All supplementary materials are available at <https://osf.io/f4y5a/files/>.

modality, namely image, text, and specification, provides a narrow slice of understanding of the underlying visualization. By integrating these modalities, our multimodal representation approach provides a more comprehensive understanding. For each visualization, we construct a triplet consisting of (1) a declarative specification using the Gosling visualization grammar [11] to define the structure, (2) a pixel-based rendering generated from that specification, and (3) a text description that details the visualization in natural language. As a result, we have developed a large document collection of such triplets, forming the foundation of our data visualization retrieval system. These examples range from common visualization types to real-world applications, such as breast cancer variants in human genomes and single-cell epigenomics.

We transform raw modalities into semantically meaningful vector representations using three complementary embedding strategies: context-free grammar (CFG) embeddings, multimodal embeddings, and textual embeddings. Our CFG embeddings are tailored to grammar-based genomics visualizations, addressing genomics-specific and previously underexplored aspects such as genomic tracks, views, and multi-scale interactivity. Multimodal embeddings leverage a state-of-the-art biomedical vision-language foundation model, while textual embeddings are generated by a fine-tuned specification-to-text LLM; both capture generalized insights from large-scale training data. In particular, with the fine-tuning approach, we employ a pretrained textual embedding model and train it on a curated dataset of text-specification pairs. This bridges the domain gap between natural language and structured representations while preserving the broad semantic capabilities of the model for accurate cross-modal retrieval.

After building multimodal representations for both queries and the document collection, we perform multimodal retrieval using similarity modeling. We use top-k retrieval accuracy as our primary quantitative metric. Notably, although the CFG approach does not require training, it achieves results comparable to the other machine learning-based methods. We conducted an informal user study with domain collaborators to gather feedback on the usability of the tool and identify improvements for better integration into their workflows. In summary, the contributions of this paper are threefold:

- First, we propose a retrieval system for interactive genomics visualizations, enabling multimodal searches by example images, natural language, or grammar-based queries over a curated collection of 3,200 visualizations across approximately 50 categories.
- Second, we develop multimodal representations combining context-free grammar embeddings, multimodal vision-language embeddings, and fine-tuned textual embeddings across three modalities: text descriptions, visualization images, and Gosling grammar specifications.
- Finally, we evaluate retrieval accuracy across embedding strategies, show that the CFG embedding approach performs well with low computational cost, and gather usability feedback from informal user studies with domain collaborators.

II. BACKGROUND

Genomics data visualization is the central focus of this work. In particular, we are interested in genome-mapped visualizations, where data is mapped to genomic coordinates. More specifically, a visualization consists of one or more coordinate systems, each applying a unique layout, partition, abstraction, and arrangement of sequence coordinates [1]. A set of features that belong to the same biological entity, such as all genes, all expression levels from the same sample, or all mutations, is called a feature set. A track is defined as a visual representation with one or more parallel genomic axes depicting one feature set. A view refers to a set of one or more aligned tracks, with at most one two-axis track. A genomics visualization can have multiple tracks nested within one or multiple views.

Authoring genomics visualizations is seen as a challenging task given the complexity of visualizations and interactions that are frequently used (e.g., coordinated many-view visualizations) [12]–[14]. A previous interview study with domain experts showed that they use various approaches for authoring genomics data visualizations [3]. One common approach is to use genome browsers, such as IGV.js [15] and JBrowse 2 [16]. While these tools enable easy creation of visualizations, they offer limited customization of visualizations to meet users’ diverse needs. Another approach is to use a visualization grammar, such as Gosling [14] and its Python library, Gos [17]. Gosling provides flexibility in authoring custom visualizations while still enabling ease of use for domain experts with computational skills, such as computational biologists and software engineers. More recently, the grammar has been deployed in Blace [18] by combining familiar graphical user interfaces with an LLM to further lower barriers for customizing genomics data visualizations.

Compared to more general visualization grammars such as Vega-Lite [5], Gosling offers specialized mechanisms tailored for effective visualization of genomics datasets. These features include native support for genomics data formats, handling of tracks and views, and multi-scale navigation. Besides computational infrastructure, genomics visualization also requires specialized visual representations that are highly domain-specific. For example, a circular layout often shows chromosomes as an ideogram along the arc, with chords connecting genomic locations that share certain biological relationships. These capabilities are not fully addressed in Vega-Lite. On top of visual representations, coordinated interaction is an important component for synchronizing navigation across multiple tracks and views; support for which is limited in general-purpose authoring tools.

III. RELATED WORK

A. Representations of Visualization

Visualization is the visual representation of data, and in this section, we explore methods for representing visualization *itself* and view visualization as the document to be represented. These representations range from intuitive formats, such as images (pixel-based displays) and text descriptions, including alt-text [19], to more structured formats, such as SVG [20] and

XML [21]. Among these, declarative grammar specifications such as Vega-Lite [5] have emerged as a particularly powerful framework for visualization design, offering a structured and expressive paradigm that forms the central focus of our work. Beyond general-purpose tools, domain-specific toolkits have been developed, including Gosling [11] for genomics and GoTree [22] for phylogenetic trees.

Beyond serving as specification languages, visualization grammars also enable representations of charts in computational analyses and machine learning (ML) workflows. Recent work has leveraged context-free grammars (CFGs) for this purpose, notably through Grammar Variational Autoencoders (GVAE) [23] with Chartseer [24], which makes use of Vega-Lite [5] JSON specs to create one-hot encodings, where it captures the presence or absence of each grammar rule. A more detailed explanation of one-hot encoding will be provided in Section VI-A3. GoTreeScope by Li and Yuan [22] applied a similar approach to tree visualization with GoTree specifications [25]. GraphScope [26] quantified transition costs among different chart specifications and visualized embeddings as nodes and the transitions as edges. ComputableViz [27] supported operations on multiple visualization specifications, enabling the generation of visualization embeddings and accessibility of visualizations. While there are various visualization grammar options, only Gosling [14] offers multi-scale navigation and characterization of genomics tracks and views. Combined with image and text modalities, this has enabled us to build a multimodal representation of genomics data visualization.

B. Visualization Retrieval

Once visual information is represented, the next task is to retrieve it effectively. Hoque and Agrawala [6] introduced a straightforward retrieval system enabling users to search for D3 visualizations with incomplete Vega-Lite JSON grammar, leveraging the structure and explicit format of specifications. VISAtlas by Ye et al. [8] utilized embeddings in the visual interface to facilitate navigation. Erato [28] and later, Chart2Vec [9] presented approaches to retrieve visualizations in data stories supporting multi-view visualizations, and introducing declarative visualization grammar for universal chart embeddings.

While the main focus of previous work is on finding visualizations with similar visual encodings, it is also important to find documents that exhibit similar data types. VizCommender [10] took this consideration into account for searching Tableau visualizations, and most recently, VAID by Ying et al. [29] explored this direction with a grammar-based approach. Notably, KnowledgePearls [30] enables searching analysis states, independent of the underlying visualization framework. At a more abstract level, Nguyen and Gehlenborg proposed Safire [31], a similarity framework that provides comparison criteria and representation considerations for visualization retrieval.

Retrieval becomes truly beneficial when we have a substantial collection. Previous work on visualization databases and corpora [32], [33] inspired us to build our own system in

the genomics domain. In particular, Olio by Setlur et al. [21] facilitates semantic search on data repositories, emphasizing the ‘pre-authored’ aspect of visualizations, which aligns with our motivation for authoring in this work.

Considering the advantages of various query formats from previous literature, our exploratory work in genomics visualization supports all three query types: image, text, and specification, to enable robust *multimodal* search. A related system, WYTWYR by Xiao et al. [7], enables retrieval with image and text inputs for general-purpose visualization using four primary attributes: *Type*, *Trend*, *Color*, *Layout*. In contrast, genomics data requires different priorities, such as *Data Type* and *Interaction*. Our pipeline also features a dedicated component for searching with specifications, extending functionality beyond that of WYTWYR.

Multimodal visualization search is a specialized case of multimodal retrieval, aiming to retrieve relevant information across heterogeneous data types. Previous surveys by Wang et al. [34] and Liang et al. [35] provide comprehensive overviews of state-of-the-art multimodal retrieval and multimodal machine learning methods, including contrastive learning [36], joint representations [37], and large-scale pretraining [38]. Building on these foundations, our work adapts and extends multimodal retrieval techniques to address the challenges of genomics visualization.

IV. METHODOLOGY

A. Design Considerations

The primary goal of the proposed data visualization retrieval system is to explore effective retrieval of genomics data visualizations by supporting diverse user needs and search strategies. Our workflow draws inspiration from prior work on encoding grammar-based visualizations [22], [24] and visualization retrieval [6], [7], [9], [10], [29], each of which employs a different form of query input. Geranium uses Gosling [14] to represent genomics visualizations, as the grammar offers built-in mechanisms for handling various genomics data formats and requirements, as described in Section II and situated within the broader landscape of visualization grammars in Section III-A.

We present the key design considerations that guided the development of the proposed system. These considerations are based on the insights gained from our formative user interviews [3], which explored existing challenges for authoring genomics data visualizations and approaches to overcome them. The participants (n=20), with diverse roles (e.g., researchers, students, software engineers) and personas (e.g., biologists, computational biologists), who have varied levels of genomics visualization experience (beginners, intermediates, experts), reported that they frequently search for existing visualization examples and reuse them to overcome their difficulties in creating visualizations. Notably, participants often employed multiple modalities, and three popular ones were (1) images of visualizations (e.g., paper figures), (2) source code for creating visualizations, and (3) textual descriptions (e.g., using Google Scholar). More importantly, participants adopted both cross-modality searches and within-modality searches (e.g., keyword-based search to find visualization images). These

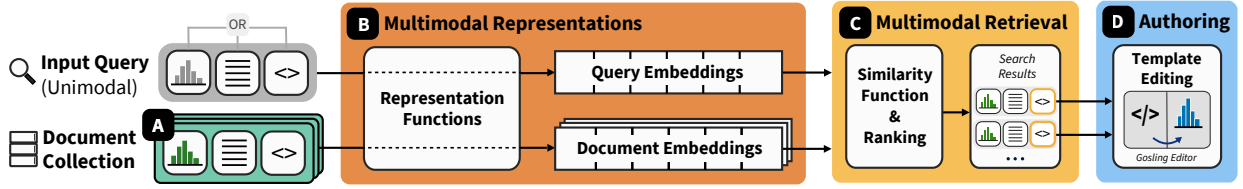


Fig. 1. Overview of the data visualization retrieval system. Panel (A) depicts the document collection, where each document is represented by a triplet of modalities: image, text, and specification, as discussed in Section V. An input query can be based on one of these modalities. Panel (B), detailed in Section VI, presents the multimodal representation, utilizing embedding methods as transformation functions to convert raw data modalities into corresponding embeddings. Panel (C) is covered in Section VII and demonstrates computing similarities between query and document embeddings to rank documents by relevance, with results presented as a list of ranked triplets. Panel (D) presents the authoring process, with the Gosling code from the retrieval results ready to be edited directly in the editor.

insights led to the three design considerations in this paper, which aim to help users more efficiently and accurately retrieve visualizations through Geranium, eventually helping their visualization authoring process.

D1. Multimodal Retrieval: Provide multimodal retrieval support to accommodate diverse user needs. The system needs to support multiple retrieval modalities to serve users with varying expertise and preferences. Users should be able to search the visualization collection through three methods: (1) example images, where users provide visual references; (2) natural language queries, allowing intuitive text-based descriptions; and (3) grammar-based queries using Gosling specifications. This flexibility aims to enable users to search regardless of their expertise or communication preference.

D2. Multimodal Representation: Construct multimodal representations that provide comprehensive information. Appropriate representations provide the basis for effective retrieval. For each visualization in the collection, it is critical to generate a comprehensive multimodal representation comprising three components: (1) a specification component using declarative Gosling grammar; (2) an image component representing the pixel-based rendering; and (3) a text component providing natural language descriptions of the visualization. These representations capture complementary information, addressing potential gaps that might exist if relying solely on any single modality.

D3. Multi-Strategy Support: Support investigation of multiple embeddings and retrieval strategies This exploratory work aims to establish a baseline and lay the groundwork for multimodal retrieval of genomics data visualizations. Therefore, it is crucial that the system supports experimentation with a range of approaches. Specifically, it needs to support both unimodal and cross-modal embedding and retrieval strategies to capture complementary information across modalities and to enable the identification of the most promising approaches suitable for genomics data visualizations.

B. System Overview

An overview of our data visualization retrieval system is presented in Figure 1. The user can begin with a unimodal query for genomics data visualizations using their preferred approach: example images, natural language, or grammar-based queries. The subsequent pipeline is operated as follows:

Document Collection (Panel A): We build the document collection of genomics data visualizations, where each docu-

ment is represented by a triplet of three modalities: image, text, and specification, in the form of (i_k, t_k, s_k) . This multimodal approach facilitates comprehensive understanding of a visualization and forms the basis for subsequent retrieval processes, which will be described in detail in Section V.

Multimodal Representation (Panel B): For each modality, we utilize embedding approaches to transform this raw data into vector representations. Our approach integrates three embedding techniques: context-free grammar embeddings, multimodal embeddings, and textual embeddings, to combine specialized knowledge from the grammar structure and general knowledge from a multimodal biomedical foundation model and a large language model. Further details of the embedding methods are discussed in Section VI.

Multimodal Retrieval (Panel C): This component involves computing similarities between the query embedding and each document’s embedding. The system uses these similarity scores to rank how relevant each document is to the search query. Documents with higher scores are considered more relevant and appear higher in the search results, which are ultimately presented as a ranked list of document triplets; Section VII presents this component in greater detail.

Authoring (Panel D): The output of the retrieval stage serves as the input for this step. Using the Gosling editor, which is pre-populated with the code from each search result, the user can directly modify this code to author new visualizations. Section VIII discusses this process in more detail within the context of the user interface.

Our integrated system allows flexible and efficient retrieval of genomics data visualizations across various query types. To evaluate the performance of this approach, we conducted experiments with top-k retrieval accuracy, along with an informal user study. These findings are outlined in detail in Section IX and discussed in Section X.

V. DOCUMENT COLLECTION

A. Approaches

Each Gosling specification can be represented as a parse tree, generated by a set of context-free grammar rules. This presents two possible approaches for data generation: (1) a top-down approach, deriving specifications from the grammar by substituting symbols with literal values, or (2) a bottom-up approach, constructing the dataset from existing example specifications.

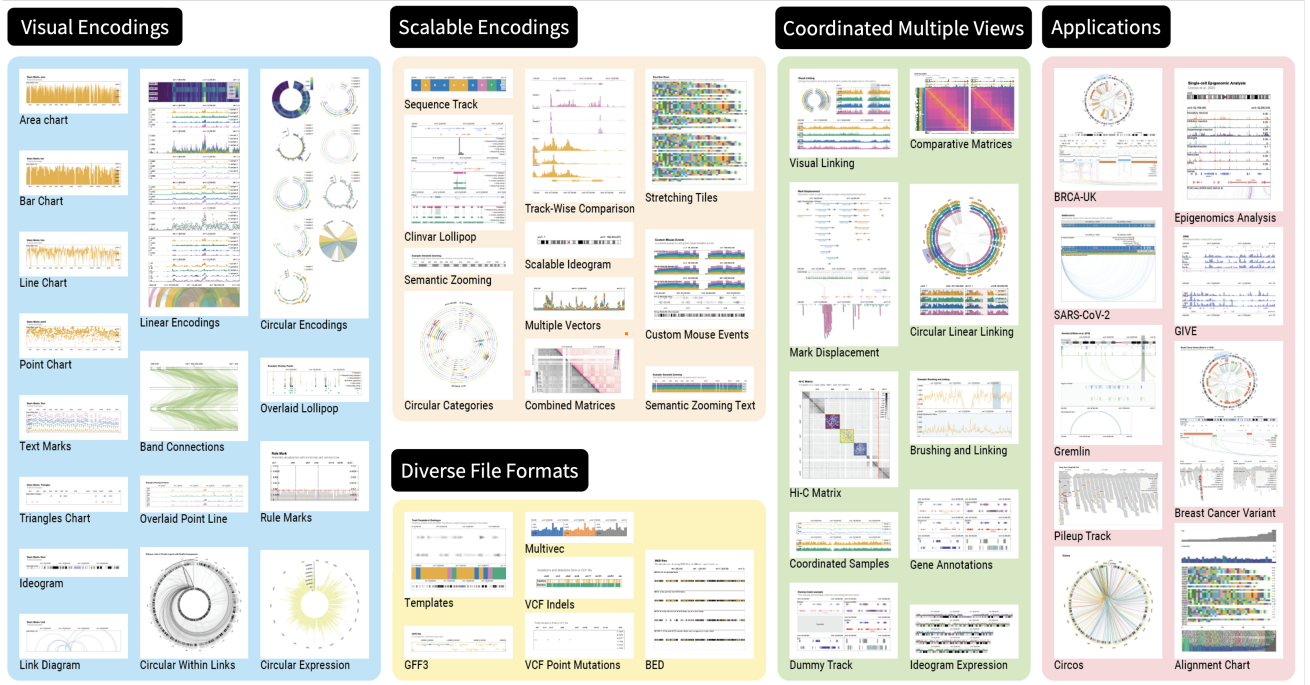


Fig. 2. The approximately 50 categories of genomics data visualizations that we collected. These categories range from common visualization types to real-world applications, such as breast cancer variants in human genomes [39].

The primary advantage of the top-down method is its comprehensive coverage of all grammar-possible scenarios. However, this approach presents significant drawbacks: the generated specifications often do not accurately reflect real-world genomics data requirements, and it produces numerous rules, many of which are redundant or unnecessary, requiring substantial effort to filter negative examples and resulting in sparse vector encodings where examples utilize only small portions of the possible rule space.

The bottom-up approach, starting with specific examples and extracting only rules present in the collection, grounds the system in actual specifications rather than theoretical possibilities. While this might miss some valid but unobserved constructions, we can mitigate this limitation by referencing the genomics visualization taxonomy by Nusrat et al. [1] to ensure coverage of essential rules. After careful evaluation of trade-offs, we selected the bottom-up approach for its practical advantages in our genomics visualization context.

B. Collecting Gosling Specifications

We collect Gosling specifications from diverse sources, including Gosling documentation and gallery examples¹, training datasets for image analysis algorithms [40], and examples from an interactive multiscale visualization for structural variation in human genomes [39]. This collection represents a wide range of examples and real-world applications that employ Gosling for genomics data visualization. In this work, we utilize the Gosling declarative grammar [14] to structure our specifications since it enables users to create expressive

visualizations while offering robust file format handling, which is an essential feature in genomics data analysis pipelines.

We collected seed examples across approximately 50 different categories, ranging from basic marks (e.g., line chart, bar chart) to domain-specific visualizations such as single-cell epigenomics and structural variation analysis, to real-world applications such as breast cancer variants and SARS-CoV-2 (see Figure 2).

For each category, we augment the seed example to create variations with different visual channels. Augmentations include permutations of mark types (such as area, bar, line, point, triangle for gene annotation; arc for genome interaction), color (categorical and continuous color maps), arrangement (vertical, horizontal, serial, parallel), layout (circular, linear), and different permutations of views within a multi-view visualization. This resulted in 3,200 visualizations.

After collecting specifications, we generated corresponding texts and images, as shown in the upper panel of Figure 3. Having the specification as the single source of truth has several advantages. First, as all generated texts and images derive their content from the same specification, it guarantees consistency and uniformity between all outputs. Second, this enables efficient updates once the specification is changed, minimizing discrepancies. An example of a visualization triplet is shown in the lower panel of Figure 3. This triplet consists of a Gosling specification in JSON format, its corresponding visualization image, and a text description. The description notes that a linear bar chart displays multivec data for chromosome 1 across four color-coded samples, with genomic positions and peak values labeled on the axes and detailed in a tooltip. The details of generating texts and images are outlined in the following Sections V-C and V-D.

¹<https://gosling-lang.org/examples/>

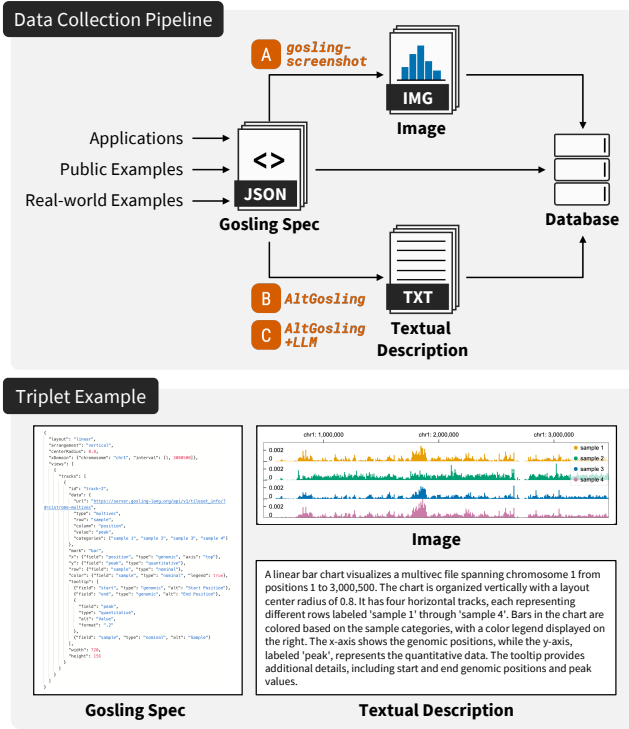


Fig. 3. Data collection pipeline and visualization triplet example. *Upper panel:* The collection of Gosling specifications in three modalities. Gosling specifications from a wide range of examples and real-world applications are collected. (A) We generated images from the Gosling specifications using an open source script, i.e., `gosling-screenshot`. Two versions of textual descriptions are constructed using (B) AltGosling [19] and (C) in combination with an LLM. *Lower panel:* Visualization triplet example, including a Gosling specification, its corresponding image, and a textual description.

C. Generating Text Descriptions

We generated text descriptions from Gosling specifications using two approaches (Figure 3B–C). First, we used AltGosling [19], a grammar-based tool which creates alt text descriptions for Gosling visualizations. This tool was originally aimed at improving accessibility for people who are blind and visually impaired, describing visual elements and providing statistical summaries of the displayed data. To generalize the descriptions, we generated a second set of descriptions with an LLM based on the AltGosling descriptions, the specifications, and images. We then developed corresponding embedding methods and evaluated the accuracy for each approach.

1) *AltGosling:* We used AltGosling (v0.2.4) to generate text descriptions for all visualizations. AltGosling fetches both an initial alt text and subsequently updates this with retrieved data. Because the purpose of our text descriptions is to query the visualization primarily, we opted to retrieve the initial alt text without fetching the underlying data of the visualization. We set up a pipeline for bulk generation of processed specifications and alt texts. The lengths of the texts varied with the number of tracks and views (mean: 1691 characters, standard deviation: 2169, median: 563).

2) *LLM-integrated Approach:* For the task of visualization querying, we hypothesized that combining AltGosling texts with an LLM would create more robust descriptions, i.e., having better query similarity for a range of queries. AltGosling

has knowledge of the underlying structure of the visualization, and in an earlier evaluation outperformed state-of-the-art LLM models in accuracy. LLMs excel in structured writing and diversity of text usage.

We queried GPT-4o [41] with the Gosling specification, an AltGosling compiled version of that specification with runtime-computed details, the automatic alt text from AltGosling, and the image. To provide context on the specifications, we also classified all properties of the specification, extracted from the CFG extraction process in Section VI, into one or more of the following classes: 1) data-related, 2) visual-encoding, 3) interaction-related, 4) styling, and 5) metadata, and added this classification to the query.

To improve consistency among text descriptions, we used a few-shot learning approach [42]. We constructed four ideal example descriptions, for which we added the specifications, AltGosling text, and ideal description to each query (omitting the image due to exceeding the maximum query length). The details of few-shot examples and prompt are included in Supplementary Material. The lengths of the generated texts were shorter and more consistent than the AltGosling-only approach (mean: 839 characters, standard deviation: 298, median: 775).

D. Generating Images

For efficient generation of images, we developed a batch image generation pipeline, based on a previously built open-source tool for single image input (`gosling-screenshot`²). This pipeline automates the process of converting multiple Gosling JSON specifications into image files. For each file, the script uses a headless browser to render the visualization in a virtual environment and captures a screenshot of the result. For technical details, see the Supplementary Material.

VI. MULTIMODAL REPRESENTATIONS & EMBEDDING METHODS

Now that we have the raw data for each modality, we will discuss three primary embedding approaches to transform these raw modalities into vector representations, supporting design consideration **D2. Multimodal Representation**. To formalize our approach, let $\mathcal{D}$ be the document collection:

$$\mathcal{D} = \{v_k \mid k = 1, 2, \dots, N\}$$

where each visualization $v_k \in \mathcal{D}$ is represented as a triplet:

$$v_k = (i_k, t_k, s_k),$$

consisting of an image $i_k \in \mathcal{I}$, text $t_k \in \mathcal{T}$, and specification $s_k \in \mathcal{S}$ for the k -th item. These modalities are semantically aligned, all describing the same underlying concept of v_k .

We propose three embedding approaches, as described in Figure 4: Context-free grammar embeddings for specifications, a fine-tuned text embedding model for texts and specifications, and BiomedCLIP multimodal foundation model for all three

²<https://github.com/gosling-lang/gosling-screenshot>

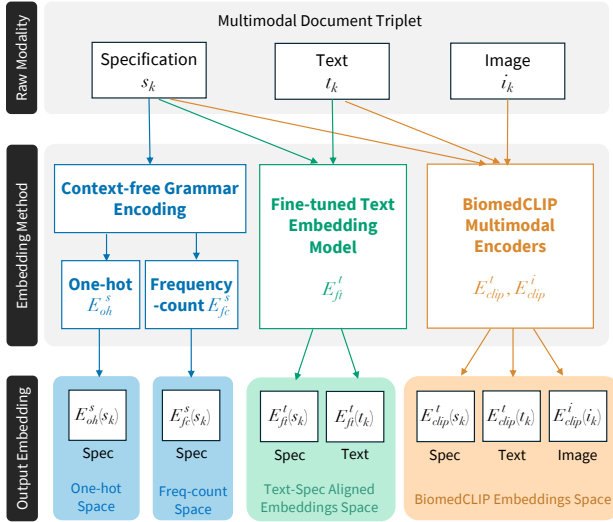


Fig. 4. Different strategies for creating embeddings from raw data for each modality. Three approaches are: Context-free grammar encoding applied to specifications, fine-tuning a general text embedding model applied to specifications and text descriptions, and a state-of-the-art foundational model BiomedCLIP for creating embeddings for all three modalities. These three approaches are described in Sections VI-A, VI-B, and VI-C, respectively.

modalities. These three approaches are described in the following Sections VI-A, VI-B, and VI-C.

A general notation for an embedding function E transforming raw data into an embedding can be defined as follows:

$$E_{method}^{modality}(input) \in \mathbb{R}^{dimension} \quad (1)$$

where *modality* specifies the types of input (image, text, or specification), *method* refers to the specific technique used to create embedding, *input* is the actual data being transformed, and *dimension* is the dimensionality of the vector space.

A. Context-free Grammar Embeddings for Specifications

To vectorize Gosling specifications, we use a context-free grammar (CFG) parsing approach, which is a formal system in which production rules define how symbols combine recursively. A production rule has a left-hand side (LHS) representing a single element, and a right-hand side (RHS) defining how that element can be expanded, which applies regardless of the surrounding symbols (hence the name “context-free”). This approach, adapted from ChartSeer [24] and the Grammar Variational Autoencoder (GVAE) concept [43], effectively captures the hierarchical structure (e.g., a `track` containing `mark` and `size`) while enabling systematic extraction of grammar patterns.

The CFG approach for creating embeddings from Gosling specifications is presented in Figure 5. Panel (A) shows a highly abstracted Gosling specification. The full specification can be seen in the Cytoband example³ on the Gosling editor. The corresponding image of the full specification is shown in Panel (B). In this example, there are five parallel chromosome views, listed from one to five in Panel (A). There are two

tracks in view (1), each with a different dataset. The first track features a multivec dataset, and the second track includes a CSV dataset containing cytogenetic band data. Under the second track, there is a nested track for band visualization, including three different mark types: `rect`, `triangleRight` and `triangleLeft`, for visualizing the ideogram.

The original algorithm effectively handled hierarchical object nesting but was not able to process arrays of unnamed elements, which is crucial for representing collections like tracks or views in Gosling specifications. This limitation is significant because genomics visualizations typically comprise multiple views, each containing multiple tracks. While traditional CFG rules work with nested objects using named properties, array elements lack explicit identifiers to serve as non-terminals in subsequent rules. As shown in Figure 3(A), a singular ‘view’ within an array has no name to reference it in following production rules. To address this, we developed an improved algorithm to explicitly manage this structural characteristic, described below.

1) *Internal Node Insertion*: We propose a simple yet effective method for adapting CFG extraction to handle arrays of enumerable elements. Our improved algorithm introduces *internal nodes* that represent individual, unnamed elements within arrays like `views`, `tracks`, and `dataTransforms`. Starting with `root`, we establish the first rule: `root` $\rightarrow$ `views` + `layout` + `arrangement`. When encountering the `views` array, our algorithm applies internal node insertion as illustrated in Panel (C) of Figure 5, where named nodes appear in solid blue color and previously unnamed nodes appear hollowed. An unnamed node (Panel C1) is assigned the name *view* (Panel C2), using the singular form of its parent node’s name (`views`). This newly named node (in solid green) is then inserted into the list of non-terminal symbols and continues in the production rule chain, starting with `view` $\rightarrow$ `tracks` + `xDomain`, for example, as in Panel C2.

2) *Extracting CFG Rules*: The extraction of CFG rules follows a systematic approach to convert visualization specifications into structured grammatical rules while preserving semantic meaning. Panel (D) of Figure 5 represents a snippet of the CFG rules extracted from the specification in Panel (A). For production rules where the RHS is a defined value accepted by the LHS in the grammar, the RHS is kept intact, e.g., `arrangement` $\rightarrow$ `"parallel"`. Otherwise, the RHS is substituted by a common token: `NUMBER`, `STRING`, similar to the approach used in [24], with a new addition of `ARRAY`. Our method parses through the entire structure of the specification, especially the `data` field (which was excluded in ChartSeer [24]), since it encodes important information about file format, a crucial aspect of genomics visualizations. Additionally, we extract production rules for *interactions*, which form an important aspect of interactive visualizations but were less addressed in previous work. These interactions include brushing via linking ID, zooming, and coordinated multiple views.

In total, we extracted 707 unique CFG rules from our document collection $\mathcal{D}$, with 205 distinct LHSs. The most complex specification in our collection contained 1633 rules. Gosling introduces 32 foundational rules to support construction of

³<https://gosling.js.org/?example=CYTOBANDS>

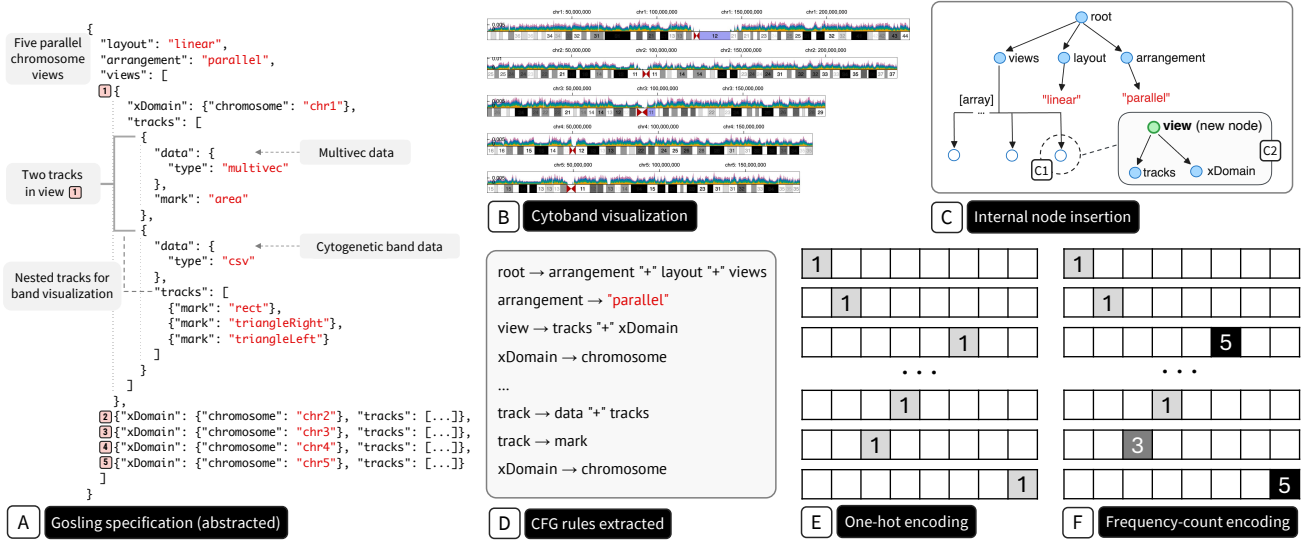


Fig. 5. The CFG approach for creating embeddings from Gosling specifications. (A) Presents abstract Gosling specification with 5 parallel chromosome views (1-5), with view (1) containing two tracks (multivec and CSV formats) and a nested track for band visualization. (B) Displays the visualization of the complete specification. (C) Illustrates algorithm improvement for handling unnamed element arrays like views, showing before (C1) and after (C2). (D) Represents a snippet of the CFG rules extract from (A). Panel (E) and (F) demonstrate the two encoding methods with CFG: One-hot and frequency-count. While the former captures the presence or absence of each CFG rule, the latter represents the prevalence by frequency of each CFG rule. Example: arrangement → "parallel" gets value 1 in both methods, while view → tracks + xDomain gets value 1 in one-hot but value 5 in frequency-count (reflecting its occurrence in each view).

visualizations according to the taxonomy by Nusrat et al. [1], which form the basis for diverse genomics data visualizations, including data types, marks, arrangements, orientations, alignments, and layouts. Our extracted CFG rules from collection $\mathcal{D}$ fully cover these 32 rules. The Supplementary Material provides a detailed description of these rules.

3) *One-hot and Frequency-count Encoding*: Panels (E) and (F) demonstrate two encoding methods using the CFG: one-hot and frequency-count. One-hot encoding captures the presence or absence of each CFG rule, while frequency-count encoding represents the prevalence of each rule by its frequency of occurrence. Previous work [9], [22], [24], including the original GVAE [43], used one-hot encoding with binary values. Additionally, we propose the use of frequency-count encoding to represent the distribution of CFG rules within the specification, better characterizing the nested structure and common patterns of these highly nested specifications.

For example, in Figure 5, rule arrangement → "parallel" occurs once, resulting in a value of 1 in both encoding approaches. However, rule view → tracks + xDomain occurs once in *each* of the five views, leading to a value of 1 in one-hot encoding but a value of 5 in frequency-count encoding. To formalize this, using notation from Equation 1 for a specification s_k and our extracted set of CFG rules:

One-hot encoding:

$$E_{oh}^s(s_k) = [u_1, u_2, \dots, u_{d_{oh}}] \in \{0, 1\}^{d_{oh}},$$

where u_i indicates the presence (1) or absence (0) of the i^{th} rule in s_k .

Frequency-count encoding:

$$E_{fc}^s(s_k) = [w_1, w_2, \dots, w_{d_{oh}}] \in \mathbb{N}_0^{d_{fc}},$$

where w_i is the count of occurrences of the i^{th} rule in s_k .

Both resulting vector spaces have $d_{oh} = d_{fc} = 707$ dimensions, as we extracted 707 CFG rules, as described in the section above. This represents a significant expansion in dimensionality compared to established visualization systems that employ CFG-based approaches. For instance, ChartSeer [24], GoTree [22], and Chart2Vec [9] each work with approximately 60 CFG rules, demonstrating their effective optimization for their respective domains. The substantial number of rules in our work reflects both the inherent complexity and expressiveness of the Gosling grammar, which was specifically designed to accommodate the rich diversity of genomics visualizations, as well as the variety present in our dataset.

B. Fine-tuned Text Embedding Model

1) *Overview*: To facilitate retrieval across text and specifications (D1), we employ textual embedding models pretrained on extensive natural language datasets. These embedding models convert textual data into semantic embeddings, enabling retrieval based on embedding similarity. Trained on large-scale textual data, these models exhibit strong generalization capabilities across diverse text types. Leveraging the semantic knowledge encapsulated in embeddings learned from vast datasets, we propose utilizing textual embedding models for effective retrieval between text and specifications.

However, in this new domain, specifications represent a novel set of rules explicitly designed for visualization, resulting in a significant domain gap relative to textual data handled by existing embedding models. To bridge this gap, we further fine-tune pretrained textual embedding models using our collected text-specification dataset. This strategy enables us to utilize the embedding models' general semantic under-

standing while effectively reducing the domain gap, making them suitable for accurate text-specification retrieval.

2) *Encoder*: Specifically, we utilize the gte-Qwen2-1.5B-instruct model [44], an embedding model derived from the Qwen2-1.5B large language model [45]. The base model employs a 28-layer Transformer architecture, where multi-head self-attention layers alternate with Mixture-of-Experts (MoE) feed-forward networks (FFNs). The query text is first tokenized and then fed into the model. Tokens are processed in an autoregressive manner, with the model generating one token at a time, each conditioned on the previously generated tokens. The final token output, derived from the full sequence of input tokens, is used as the embedding representation of the query text. The embeddings are then compared to the preprocessed embeddings to identify the most relevant data.

3) *Loss Function*: We fine-tune the model using the multiple negatives ranking loss (MNRL) [46] with paired text-specification data. Positive pairs are extracted from our dataset by pairing each specification with its corresponding descriptive text. The MNRL for a single example is defined as:

$$\mathcal{L} = -\log \left(\frac{\exp(\text{sim}(\mathbf{a}, \mathbf{p}))}{\sum_j \exp(\text{sim}(\mathbf{a}, \mathbf{p}_j))} \right),$$

where $\mathbf{a}$ is the embedding of the *anchor* (e.g., a sentence), and $\mathbf{p}$ is the embedding of its corresponding *positive* (e.g., a specification). The index j ranges over all positive samples in the batch. The similarity function $\text{sim}(\mathbf{a}, \mathbf{p})$ is the cosine similarity:

$$\text{sim}(\mathbf{a}, \mathbf{p}) = \frac{\mathbf{a} \cdot \mathbf{p}}{\|\mathbf{a}\| \|\mathbf{p}\|} \quad (2)$$

This loss encourages each anchor to be more similar to its own positive than to any other positive in the batch, which are treated as implicit negatives. As a result, it improves the model's ability to retrieve the correct specification given a query sentence. We adopt MNRL because it leverages in-batch negatives without requiring explicit negative mining, thus producing a stronger and more stable training signal than triplet loss, which depends on hard-negative selection and margin tuning.

4) *Training Settings*: We load the pretrained model weights from the gte-Qwen2-1.5B-instruct model and further fine-tune the model with our collected text-specification data pairs. All model weights are fine-tuned with no frozen layers to ensure adaptation to the text and specification domain.

We train the model with a batch size of 256 for 8 epochs using the AdamW optimizer [47] and a cosine learning rate scheduler with a warmup ratio of 10%. The initial learning rate is $2e-5$.

The fine-tuned text embedding model provides a unified function $E_{ft}^t(x) \in \mathbb{R}^{d_{ft}}$, where x can be either text t_k or specification s_k , with $d_{ft} = 1536$, embedding both text and specifications into the same vector space and allowing direct comparison between these two modalities.

C. BiomedCLIP Multimodal Encoder

In this section, we utilize semantic search using vector embeddings. We select the multimodal biomedical foundation model BiomedCLIP [48] to generate embeddings for all three modalities: images, text, and specifications within a shared semantic space (**D2**). The BiomedCLIP embedding space is trained on a vast dataset of 15 million biomedical image-text pairs, extracted from 4.4 million articles. BiomedCLIP uses a vision encoder for images and a text encoder for both text and specifications (which are treated as raw text), resulting in a unified embedding space where representations of all three modalities are directly comparable.

The image encoder transforms an input image, i_k , into the corresponding embedding:

$$E_{clip}^i(i_k) \in \mathbb{R}^{d_{clip}}$$

The text encoder processes both text inputs, t_k , and specification inputs, s_k , generating their respective embeddings:

$$E_{clip}^t(t_k) \in \mathbb{R}^{d_{clip}}, \text{ and } E_{clip}^s(s_k) = E_{clip}^t(s_k) \in \mathbb{R}^{d_{clip}}$$

where $d_{clip} = 512$.

As stated in design consideration **D3**, we aim to experiment with different strategies. Through approaches such as a fine-tuned text embedding model or the BiomedCLIP vision-language model, we seek to test whether the raw format of specifications is a promising approach compared to the more feature-engineered approach used in CFG. For an intuitive view of the embedding spaces, UMAP visualizations of all embeddings are included in the Supplementary Material.

VII. MULTIMODAL RETRIEVAL

A. Retrieval Process

1) *Overview*: Our retrieval process involves two main steps. First, we transform the input query $q_j^{(m)}$ using the appropriate embedding function for its modality $m \in \{i, t, s\}$, as demonstrated in Section VI. We then compute similarities between this query embedding and document components using the similarity modeling strategies outlined in Section VII-B. These similarities are used to rank all N documents in collection $\mathcal{D}$, with higher scores indicating greater relevance.

A key feature of our system is that it always retrieves a ranked list of complete triplets for each result, regardless of the modality or embedding space used for similarity computation. This is achieved due to the system design, where each database entry is a triplet of text, image, and specification referring to the same underlying visualization. This alignment makes sure that, regardless of the input query modality (text, image, or specification), once a relevant document is retrieved from similarity modeling, the other modalities can be directly accessed as part of the same triplet. This cross-reference step leverages the pre-aligned nature of our dataset and avoids the need for explicit embedding aggregation or direct mapping between different embedding spaces. These system designs are to support design choice **D1. Multimodal Retrieval**, accommodating diverse query preference. Our multimodal

		Document Modality		
		Specification	Text	Image
Query Modality	Specification	One-hot	Text Model	BiomedCLIP
		Frequency-count		
		Text Model	BiomedCLIP	
		BiomedCLIP		
	Text	Text Model	Text Model	BiomedCLIP
		BiomedCLIP	BiomedCLIP	
	Image	BiomedCLIP	BiomedCLIP	BiomedCLIP

Fig. 6. Modality pairings for query (rows) and document (columns). For specification, four different embedding methods can be applied: One-hot encoding, frequency-count encoding, fine-tuned text model, and BiomedCLIP, with the latter two processing specifications as raw text. For text, the applicable methods are the Fine-tuned Text Model and BiomedCLIP, while for images, BiomedCLIP is used.

retrieval system also enables both intra-modality and cross-modality comparison strategies, as illustrated in Figure 6. This figure shows the possible combinations of modalities for query input (rows) and document (columns) comparison.

2) *Method-Specific Embeddings*: The retrieval accuracy reported for each method is based on its own embedding space, where the differing embedding sizes reflect the native design choices of each model. This is to support design choice **D3. Multi-Strategy Support** for different data types: (1) Specification embedding (CFG) produces vectors of a size corresponding to the number of dimensions we characterize in a Gosling specification; (2) BiomedCLIP, being a specialized vision-language model, uses its own standard embedding dimension; and (3) the fine-tuned text model has its own inherited, model-specific embedding dimension.

For all our experiments, we conduct comparisons at the *method* level, not the embedding vector level. Retrieval and similarity search are always performed within the embedding space of the respective method, as later shown in Table I, and thus does not affect fairness. For example, text queries are embedded and compared only using the text embedding model, specification queries within the specification embedding method, and so on. Therefore, the embedding size of each model is fixed throughout its own retrieval process, as we do not compare similarities across the embedding vectors of different models.

B. Similarity Modeling

1) *Intra-modality Similarity*: The most straightforward approach occurs when the modality of the query matches one of the document components. In these cases, we compute similarity directly between embeddings of the same type $m \in \{i, t, s\}$, with the cosine similarity function:

$$\text{sim}_{\text{intra}}(q_j^{(m)}, m_k) = \frac{E^m(q_j^{(m)}) \cdot E^m(m_k)}{\|E^m(q_j^{(m)})\| \|E^m(m_k)\|}, \quad (3)$$

where E^m represents the appropriate embedding function for modality m . For example, a specification query is compared

directly with the specification document components using the same embedding method E^s for direct matching; in our case, it can be either: one-hot embeddings E_{oh}^s , frequency-count embeddings E_{fc}^s , fine-tuned model $E_{ft}^s = E_{ft}^t$, or BiomedCLIP text encoder $E_{clip}^s = E_{clip}^t$.

2) *Cross-modality Similarity*: While intra-modality matching is intuitive, many real-world scenarios require comparing across different modalities (**D1**). For cross-modal retrieval, we define the comparison between query modality m and target modality m' within the shared space as follows:

$$\text{sim}_{\text{cross}}(q_j^{(m)}, m'_k) = \frac{E_{\text{shared}}^m(q_j^{(m)}) \cdot E_{\text{shared}}^{m'}(m'_k)}{\|E_{\text{shared}}^m(q_j^{(m)})\| \|E_{\text{shared}}^{m'}(m'_k)\|}, \quad (4)$$

where E_{shared}^m and $E_{\text{shared}}^{m'}$ denote the embedding functions that map content from modalities m and m' respectively into a common shared embedding space. For example, when comparing a text query with specification components in a document, we can use embedding models that handle both modalities, such as our fine-tuned model E_{ft}^t or BiomedCLIP E_{clip}^t , both of which process specifications as raw text.

3) *Ensemble Approach*: Given the diversity of embedding methods and similarity computation strategies, we recognize that no single approach is optimal for all query types and documents. To leverage the complementary strengths of different methods, we compute multiple similarity scores and select the highest. As shown in Figure 6, for each input modality (row), we compare all similarity values across that row to determine the most effective retrieval approach.

$$\begin{aligned} \text{sim}_{\text{ensemble}}(q_j^{(m)}, v_k) \\ = \max\{\text{sim}_{\text{intra}}(q_j^{(m)}, m_k), \text{sim}_{\text{cross}}(q_j^{(m)}, m'_k)\} \end{aligned}$$

where each components in the max function corresponds to equation (3) and (4) in the previous sections. This ensemble approach allows us to select the most effective similarity measure for each query-document pair. For instance, when processing a text query, the system may either compare it directly with document text components via intra-modality similarity, or leverage cross-modal mappings to match against image or specification components, whichever produces the highest confidence match.

VIII. USER INTERFACE AND AUTHORING

We designed and implemented a prototype user interface that enables users to retrieve genomics data visualization examples using the three modalities. The overall interface consists of two components: Search and Authoring interfaces (Figure 7A–B). On the Search interface, the panel on the left (A1) enables users to provide three input modalities (i.e., textual descriptions, images, and Gosling [14] specifications). For example, users can describe a visualization that they wish to search for, such as data types used in visualizations or visualization types (e.g., “two linked comparative matrices using hi-c and micro-c datasets”). Using the image modality as an

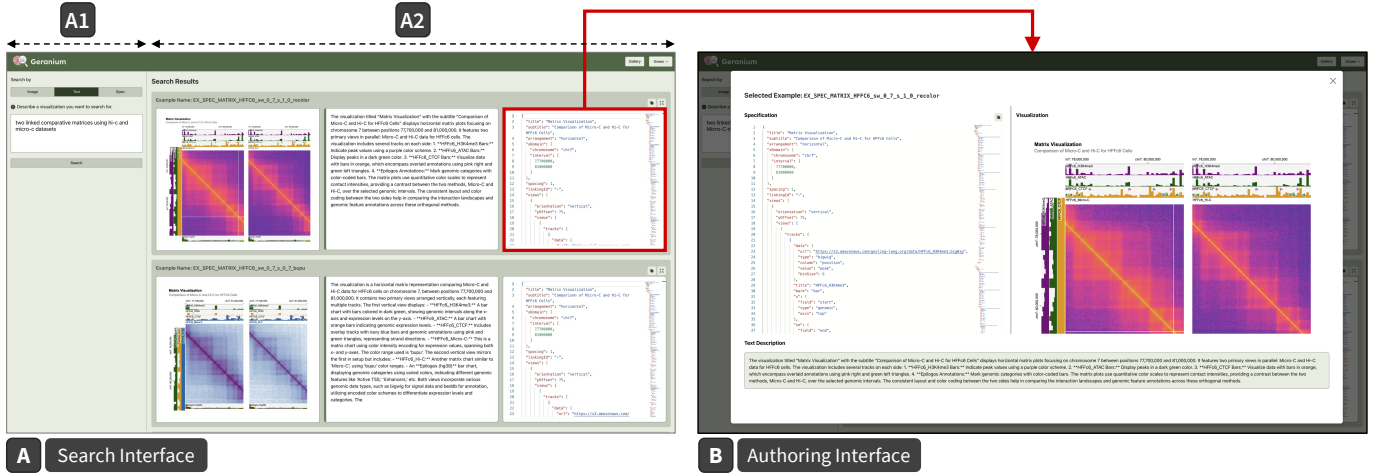


Fig. 7. The overall user interface, with (A) Search interface, including: (A1) Users can search for examples of genomics data visualization using three modalities: textual descriptions, images, and Gosling [14] specifications. (A2) The search results are shown as a gallery, displaying examples with all three modalities. (B) The user can directly modify the code of search results to author new visualizations.

input, users can alternatively use an actual image, such as a publication figure. Users can also directly upload (or copy and paste) a Gosling specification, such as a partial specification that contains a visualization that users wish to use in addition to other visualizations. Once users press a Search button, the panel (A2) on the right displays the visualization examples based on the input query. The results are shown as a gallery with visualization thumbnails, enabling easy visual exploration [49]. All other modalities, Textual Descriptions and JSON Specifications, are displayed together, which helps users to find the right visualization example that they are looking for. For example, the textual description provides additional details of a given visualization, such as the datasets used and user interactions supported. The three modalities can be shown in detail in a dedicated pop-up view on demand (i.e., upon clicking on a single example of interest, the corresponding image, textual description, and specification are shown in detail on a maximized view).

Once users find a visualization example, they can export its Gosling specification and use it in existing visualization authoring ecosystems (e.g., the Gosling editor⁴ or the Blace graphical user interface [18]). The landing page also includes a gallery to browse the dataset (Supplementary Material).

Using the search results as templates, the user can directly author new visualizations, as shown in Figure 7(B). By clicking on either an image or code example, a Gosling editor opens, pre-populated with the code from the selected search result and ready for editing. Within the authoring interface, the code editor is located on the left, the rendered visualization appears on the right, and a description of the original visualization is displayed along the bottom, providing additional context and information.

a) Implementation: The prototype system implements front-end for user interfaces and back-end for using ML models and storing data. The front-end is implemented in JavaScript using React 18 [50], and the back-end is im-

plemented in Python using Flask [51]. The server stores all genomics data visualizations we collected in the three modalities, as well as their embeddings. The input textual descriptions, images, and specifications from the user interface is sent to the server via Flask, and the server finds the most similar visualization examples. The back-end uses BiomedCLIP [48] and also implements the CFG rule extraction to generate the embedding of the given input. The source code is publicly available at <https://github.com/gosling-lang/geranium>.

IX. EVALUATION

A. Quantitative Evaluation

1) Experimental Setup: To evaluate the effectiveness of our system, we created a test suite and applied standard information retrieval metrics that measure the accuracy of the top- k retrieved items.

We define a set of unimodal queries as follows:

$$\mathcal{Q}^{(m)} = \{q_j^{(m)} \mid j = 1, 2, \dots, Q\},$$

where Q is the total query count. Each query $q_j^{(m)}$ retrieves its relevant target triplets (ground truth). With document collection $\mathcal{D}$, the ground truth for query $q_j^{(m)}$ is defined as: $\mathcal{G}_j \subseteq \mathcal{D}$, including the original triplet and semantically similar documents.

From the collection $\mathcal{D}$, we selected representative visualizations for querying. For singular visualizations, we removed titles and applied one random modification when possible: 1) removing the last item in the list of data samples, 2) doubling `binSize`, 3) changing axis position, 4) removing axis, or 5) removing legend. For composite visualizations, we included only one or multiple subsets. From these modified specifications, we generated text descriptions and images as in Section V, creating 204 queries per modality.

With a mapping function ϕ where visualization v_k maps to query index $\phi(k) \in \{1, 2, \dots, Q\}$, for each original visualization

⁴<https://gosling.js.org>

TABLE I

QUANTITATIVE COMPARISON OF TOP-K RETRIEVAL ACCURACIES (K FROM 1 TO 5). THE HIGHEST ACCURACIES FOR EACH QUERY MODALITY ARE DENOTED IN BOLD. DOCUMENT MODALITIES WITH ASTERISKS (*) INDICATE THE INTRA-MODALITY SEARCH. SEE THE SUPPLEMENTARY MATERIAL FOR THE FULL TABLE WITH K FROM 1 TO 10.

Query Modality	Document Modality	Embedding Method	k=1	k=2	k=3	k=4	k=5
Spec	Spec*	Frequency Count	0.6127	0.7206	0.7647	0.8235	0.8382
		One Hot	0.6127	0.6618	0.75	0.8039	0.8137
		BiomedCLIP	0.3333	0.4069	0.4510	0.4853	0.5147
		Text Model	0.5882	0.6029	0.6225	0.6373	0.6373
	Image	BiomedCLIP	0.0343	0.0343	0.0441	0.0441	0.0441
	Text (AltGosling + LLM)	BiomedCLIP	0.0294	0.0588	0.0735	0.0882	0.1029
		Text Model	0.3627	0.4706	0.5294	0.5637	0.5784
	Text (AltGosling)	BiomedCLIP	0.0294	0.0294	0.0294	0.0294	0.0294
		Text Model	0.2353	0.3186	0.4069	0.4461	0.4559
	Spec	BiomedCLIP	0.0147	0.0196	0.0196	0.0196	0.0294
Image	Image*	BiomedCLIP	0.6520	0.6863	0.7108	0.7304	0.7402
	Text (AltGosling + LLM)	BiomedCLIP	0.2108	0.3137	0.3725	0.4265	0.4657
	Text (AltGosling)	BiomedCLIP	0.1422	0.1716	0.1863	0.1863	0.1912
	Spec	BiomedCLIP	0.0392	0.0637	0.0784	0.0931	0.0980
Text (AltGosling + LLM)	Image	Text Model	0.3529	0.3824	0.4069	0.4412	0.4510
		BiomedCLIP	0.2255	0.2745	0.3088	0.3333	0.3627
	Text (AltGosling + LLM)*	BiomedCLIP	0.4510	0.5539	0.6127	0.6422	0.6765
		Text Model	0.3824	0.4755	0.5147	0.5637	0.5980
	Text (AltGosling)	BiomedCLIP	0.2206	0.2794	0.3235	0.3382	0.3480
		Text Model	0.2598	0.3627	0.4510	0.4657	0.4706
	Spec	BiomedCLIP	0.0245	0.0441	0.0441	0.0490	0.0490
		Text Model	0.1667	0.2059	0.2353	0.2500	0.2549
Text (AltGosling)	Image	BiomedCLIP	0.0784	0.1176	0.1667	0.1667	0.1716
	Text (AltGosling + LLM)	BiomedCLIP	0.2696	0.3186	0.3725	0.4118	0.4363
		Text Model	0.1961	0.2745	0.3186	0.3529	0.3824
	Text (AltGosling)*	BiomedCLIP	0.4363	0.5343	0.5784	0.6324	0.6520
		Text Model	0.2696	0.3578	0.4608	0.4804	0.4951

$v_k = (i_k, t_k, s_k)$ in $\mathcal{D}$, we created queries in three modalities: $(q_{\phi(k)}^{(i)}, q_{\phi(k)}^{(t)}, q_{\phi(k)}^{(s)})$ representing image, text, and specification queries. This preserves the connections between queries and source visualizations.

To extend the query ground truth, we identified semantically similar visualizations. If v_k derives from a seed example $v_{seed} \in \mathcal{D}$, we extend ground truth for queries $q_{\phi(k)}^{(m)}$ by including v_{seed} and any $v_j \in \mathcal{D}$ derived from v_{seed} where i_j is visually similar to i_k .

2) *Top-k Retrieval Accuracy*: For a given query $q_j^{(m)}$, if the correct answer is found within the top k returned items, then the retrieval is considered accurate. Top- k retrieval accuracy measures the ratio of instances for which the true or relevant item is included within the top k predicted results. This metric allows us to evaluate how often users will find relevant results within a reasonable number of items to review, which directly relates to the usability of the retrieval system.

To formalize this, given Q as the total number of queries evaluated, top- k retrieval accuracy is defined as:

$$A_k = \frac{1}{Q} \sum_{j=1}^Q \mathbb{I}(g_j \in R_{k,j}),$$

where g_j specifies the ground truth label for query $q_j^{(m)}$, $R_{k,j}$ represents the top k items retrieved for that query, and $\mathbb{I}(\cdot)$ is the indicator function, which returns 1 if the condition inside it is true, and 0 otherwise. In this case, it checks if the ground-truth label g_j is within the top k predictions ($R_{k,j}$).

3) *Results*: Table I summarizes the top- k retrieval accuracy (k=1 to 5) with different embedding methods. For the full table with k=1 to 10, see Supplementary Material. Overall, intra-modality queries showed higher top- k accuracies compared to cross-modality queries. Figure 8 shows the top- k retrieval accuracy for four query modalities. The image-to-image retrieval using BiomedCLIP showed the highest accuracy when $k = 1$ (0.65), while spec-to-spec retrieval using the frequency count encoding showed the best performance with higher k , reaching 0.9 with $k = 10$. Comparing two types of encoding for the context-free grammar (CFG) rules, the frequency-count encoding showed slightly better performance than the one hot encoding for all five different k options, indicating that frequency-count encoding better captures the characteristics of genomics data visualization (e.g., multiview aspects).

Compared to the CFG approach, treating specifications as textual descriptions showed much lower accuracies using both

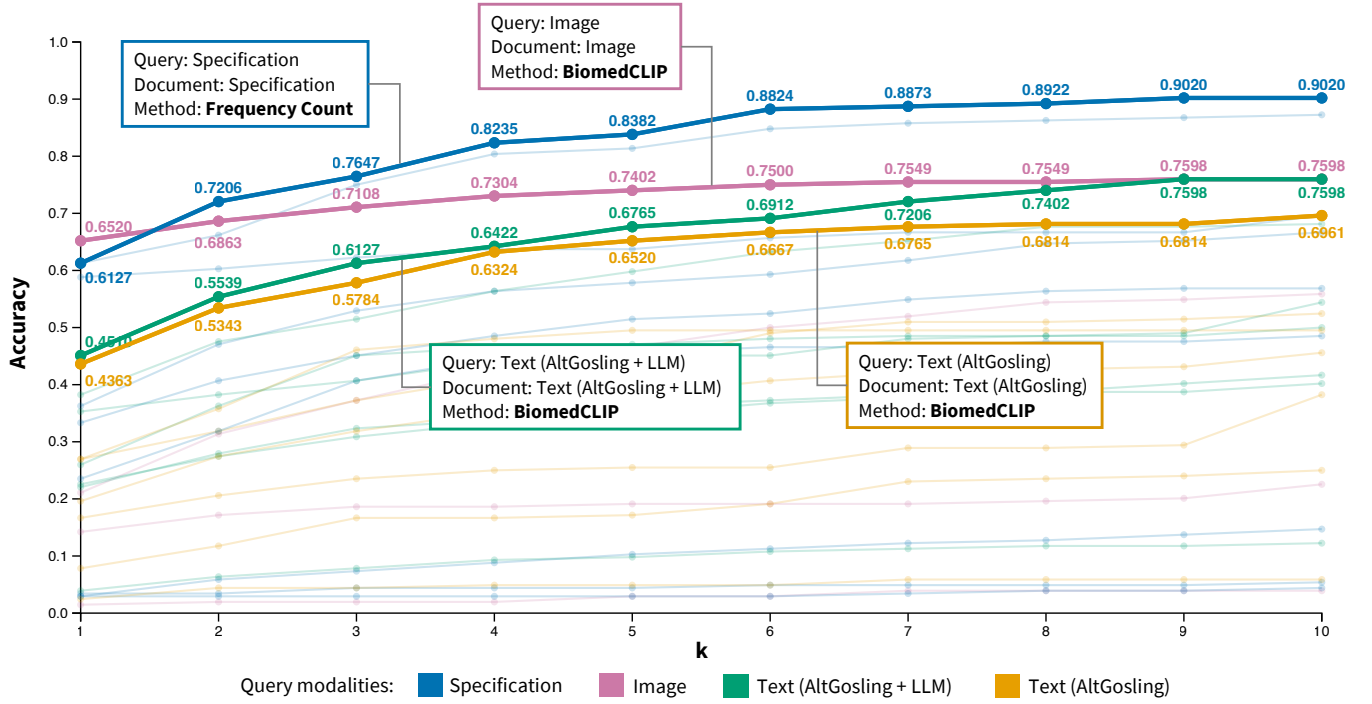


Fig. 8. Top- k retrieval accuracy for four query modalities: Specification, Image, Text (AltGosling + LLM), and Text (AltGosling). The numeric labels indicate the highest retrieval accuracy for each query modality, all of which are achieved by intra-modality searches. The specification modality with frequency-count approach shows the highest overall accuracy and is not computationally heavy. Results shown in lighter colors correspond to query-document modality combinations resulting in lower retrieval accuracy (see Table I and Supplementary Material).

BiomedCLIP and a trained ML text model. This finding aligns with the complexities inherent in the JSON specifications, as highlighted in prior studies [52]. Between two versions of textual descriptions, we found that the version with an LLM showed overall better accuracies, likely due to flexibility in word choice leading to more robust embeddings. When searching with Gosling specifications, our fine-tuned text model outperformed BiomedCLIP but was still less effective than the frequency-count method. A potential factor is that the text model depends on the raw specification strings, whereas the frequency-count approach benefits from structured features extracted from CFG rules.

For similarity score comparisons, see Supplementary Material. Multiple embeddings can have the same similarity score, in which case there is no intuitive ordering for the documents. They are returned ordered based on previous ordering, e.g. alphabetical. This influenced the top- k accuracy, and is a caveat in the document retrieval strategy. In 31.9% of queries an arbitrary ordering affected the top- k accuracy score (with similar percentages for each query modality).

B. Qualitative Evaluation: Informal User Study

1) *Study Design*: We conducted an informal user study with seven domain collaborators ($n=7$) to gather feedback on the retrieval system Geranium. The participants were researchers in biomedical informatics with experience in genomics and omics data visualization at various career stages, including three postdoctoral researchers (P3, P6, P7), two graduate researchers (P4, P5), and two undergraduate researchers (P1,

P2). Although the user sample size is relatively small, we aimed to capture a broad range of usage scenarios.

The semi-structured interview lasted about 50 minutes and followed a three-part sequence: the first 10 minutes for workflow description, the next 10 minutes for tool demonstration, and the last 30 minutes for hands-on exploration and feedback. First, participants described their workflows for searching and authoring genomics visualizations, which helped contextualize the discussion and situate them in real-world scenarios. Next, the first author, as interviewer, demonstrated key features and capabilities of the tool. For this study, we selected the embedding strategies that achieved the highest accuracies in the quantitative evaluation. Specifically, for the specification, we used frequency-count embeddings, and for both image and text we used BiomedCLIP, with AltGosling+LLM providing the text descriptions. Finally, participants explored the tool themselves, performed searches relevant to their day-to-day tasks, and were encouraged to use all three modalities (text, image, and specification) as search queries. They provided feedback on usability, usefulness, and on how the tool could better fit into their workflows. The feedback included participants ranking their query modality preferences with scores from 1 to 3 based on their likelihood of use in practice, with 3 being most preferred and 1 being least preferred.

2) *Findings: Usability and Usefulness*. Overall, participants found the tool “helpful”, with search results described as “decently well” matched to their queries and the template-based workflow as easy to “plug and play” while requiring “less building from scratch.” While the relevance of search results varied (more on this in the following sections), all

	P6	P4	P5	P3	P7	P1	P2	Avg.
Image	3	3	3	2	2	1	2	2.29
Text	1	2	2	3	3	3	1	2.14
Spec	2	1	1	1	1	2	3	1.57

Preference scores: 3 = high, 2 = medium, 1 = low

Fig. 9. Preference scores for three input modalities (image, text, and specification), as rated by seven participants (P1–P7). Each cell shows the participant’s score for that modality: 3 = most preferred, 1 = least preferred. Darker blue background indicates higher preference. The rightmost column shows the average preference score for each modality across all participants.

participants were able to search using text and images, whereas some hesitated to use Gosling specifications, as this required some familiarity with the grammar for both searching and editing. Geranium was reported to provide an easier mapping between data, specification and visualization compared to awesome-genome-visualization [53], and more control than the UCSC Genome Browser [54] (P1). The interface is “user-friendly” (P2, P4), and a participant noted “I could see myself using this to make publication figures” (P4). Participants found text descriptions helpful for understanding visualizations (P4, P5), noting it is “nice to have the text to be able to draw those comparisons” (P5) in comparative matrix views, and that “having the text description telling me generally what [the visualization] is trying to do is very helpful” (P4). However, some participants noted that the tool can feel “intimidating” (P6) for new users of the Gosling grammar and that finding the exact desired visualization requires “more effort” or “tweaking” for highly customized or uncommon plots.

The usefulness of image-based search was highlighted when users aimed to reproduce visualizations found in the wild. For example, P3 had found an interesting research paper and wondered if they could recreate a genome browser snapshot similar to the one in the article. To explore this, they captured a screenshot of the figure and dragged it into the search box as a query. Although that exact figure was not in the dataset, the system returned genome browser views with similar layouts and track arrangements. As these results were fully modifiable, users could go from static figures in the literature to customizable, interactive visualizations. This scenario is demonstrated in our video in the Supplementary Material.

User Experience and Preference with Different Query Modalities. Participants showed clear but varied preferences for query input modality with text, image, or Gosling specification. This preference often depends on the purpose of the search, the background of the users, and their familiarity with the Gosling grammar. Although there were only seven participants, the preference patterns varied greatly. Figure 9 presents the query input preferences among the seven participants, with a higher score representing a more preferable modality, where the participant is more likely to use that as the query format.

Overall, image queries are the most preferred, with the highest average score (2.29). Image search can be a good starting point: “I see a visualization in a paper that I wanna copy, [...] that would be a good place to start to just be able

to copy-paste that picture in” (P4), or it can narrow down the search more flexibly, in comparison with text: “easier to narrow in more on what you want with the image than by typing everything out in text” (P5). Searching with text can help users “quickly get started and see types” (P6), but it poses challenges due to ambiguity in user intent (P2, P3, P7). Mismatches occur between the user intent and what the engine understands (P3), or between user vocabulary and database or taxonomy alignment, e.g., “not just because of the software, but also because my description is not precise” (P7).

Preference for specification is often found among those with a certain level of familiarity with Gosling and a clear intended search result in mind (P1, P2). For example, when they try to find “a very specific element of the spec, e.g., brushing, I can search for all the visuals that use brushing” (P1), or “I can just get the exact code to customize with.” (P2). In a similar vein, P6 mentioned that they had not tried the specification, but it seemed to be something they would use “if I already know exactly what I’m looking for.” Otherwise, it can be “intimidating if you don’t already know the vocabulary.” (P6).

Balancing Variety and Similarity in Search Results. Participants mentioned the importance of seeing diverse examples in search and galleries, but also wanted relevant, precise matches near the top. In this context, top results should ideally balance similarity for precision with the inclusion of alternative visualizations for inspiration and diversity (P2, P3). In particular, P3 suggested that clustering or grouping views could help, with a representative from each family of visuals. Although not exactly what they wanted, a participant mentioned that the variety in search results “lets me think of different ways to visualize” (P6).

Gallery Browsing for Orientation. Many users gravitate toward starting in the gallery, using visual examples to get an overview of the database and clarify intent, where “the gallery can help kind of inform the search” (P7) by providing a clear indication of the scope and coverage of what the database is capable of before searching specifically (P6, P7). It is helpful for users to be able to browse the diverse gallery rather than just rely on ranked list views (P1, P2). It can also promote design inspiration, as people may “get stuck a lot in the plots they’re comfortable with, so it’s fun to see all the different options like this” (P5). The gallery can be a “starting place” where the participant can begin to look for something similar to what they want to generate. The gallery is suggested to be more useful if it has a hierarchy to let users select categories to see corresponding examples (P1).

Supporting Onboarding and User Intent. For helping users get familiar with the tool, P1 and P2 suggested having an onboarding process with Gosling instructions and a brief overview, which would be beneficial for new users. A participant mentioned it would be “fantastic” to be able to search with more than one modality at once, e.g., text plus image, or image plus partial specification, to modify or highlight certain parts of the search (P3). Enhancing the understanding of user intention is a suggested area for improvement, following issues with mismatched vocabulary (P2, P3, P7). A general observation in searching with text is that there is a tendency among participants to type full phrases or sentences (“show

me the [visualization]”) instead of keywords, as is often seen in web search engines such as Google or Bing. This may be explained by the prevalence of LLM conversations and how people communicate with such tools.

X. DISCUSSION

A. Retrieval Performance Across Modalities

This paper introduced and evaluated a multimodal database system designed to address the challenge of retrieving relevant genomics data visualizations, using multimodal representations. The ensemble similarity approach combines the cross-modality and intra-modality approaches, presenting a complete picture of the combined effort. Our evaluation, measured by top-k accuracy, reveals several key trends. Intra-modality retrieval consistently outperformed cross-modality searches. Notably, specification-to-specification search was most effective with frequency-count encoding ($A_5 = 0.84$), followed closely by one-hot encoding ($A_5 = 0.81$). Importantly, the two CFG-based encodings do not require training, yet demonstrate comparable results to methods using ML models, and are comparable to or better than the fine-tuned text embedding model ($A_5 = 0.64$), and BiomedCLIP ($A_5 = 0.51$), where specifications are treated as plain text. In general, the foundational pre-trained BiomedCLIP model outperformed our fine-tuned text embedding model, indicating that fine-tuning on our current corpus does not yet outweigh the benefits of general-purpose pre-trained models. For text queries, AltGosling+LLM descriptions offer an advantage, where the delineation of specific classes (data, visual-encoding, interaction, styling, and metadata) also helps steer the LLM in the right direction. Since the improvement associated with the LLM-augmented method was small, further investigation is needed for a detailed assessment of the benefits and drawbacks including cost, as reported in the Supplementary Material, of incorporating LLM support. Comparing our results to Chart2Vec [9] ($A_2 = 0.63$, $A_5 = 0.73$), our approach ($A_2 = 0.72$, $A_3 = 0.76$) with frequency-count encoding performs slightly better, while not requiring any training effort.

B. Diverse Usage Scenarios and Preferences

In the user study, we observed two main tendencies in how participants conducted searches. Some began with a particular query in mind and aimed to find specific examples, while others searched more broadly with an exploratory goal. In most scenarios, postdoc and graduate researchers searched for visualizations of data types they already worked with. In contrast, undergraduate researchers more often focused on finding examples that offered certain encodings or user interactions, to explore the design space and identify feasible options. Participants also emphasized that top results should ideally balance close matches for precision with alternative visualizations that provide diverse inspiration. This aligns with prior work on active search with and without purpose in design inspiration [55] and in visualization specifically [56]. Active search with purpose refers to searching deliberately with a specific goal, whereas active search without purpose is not driven by a specific intention. These patterns offer

initial evidence that multimodal retrieval can support a range of user needs beyond providing alternative input methods. While the small user sample limits the generalizability of our results, our findings offer a motivation for further research on queryable resources to support discovering visualization design inspirations.

From a technical perspective, although the specification modality has the highest top-k retrieval accuracy $A_{10} = 0.9$, it is the least likely to be used (average score 1.57, Figure 9) since it requires users to have a certain level of familiarity and comfort with using Gosling. On the other hand, image-based search was the most preferred and exhibited comparable accuracy $A_{10} = 0.76$ (Figure 8). Our quantitative and qualitative evaluations indicate that text-based search is less accurate than other modalities and more prone to intent ambiguity. This highlights that different query modalities, such as image-based and code-based search, can lead to significantly different results. This finding on *query modality* complements the finding from Safire [31], where *representation modality* influences retrieval capabilities.

C. Practical Deployment and Generalizability

The prototype described in this work relies on visualizations rendered with their original published data. A practical deployment or broader application, however, might require ingesting visualizations paired with potentially synthetic datasets or representative examples rather than data from source publications. The specification results from this search can be used as scaffolds for authoring, allowing users to directly modify the specifications and fill in their own data and styles. Our study observed this in practice: participants moved between gallery browsing, image and text searches, and specification editing at different authoring stages, suggesting the three modalities serve complementary roles within a single authoring workflow.

Although our system is developed primarily for genomics data visualizations, the approach can be extended to other domains. The CFG-based approach is domain-agnostic and applicable to other domain-specific languages expressible as JSON-based grammars, such as Vega-Lite and GoTree. In particular, the Internal Node Insertion algorithm described in Section VI-A1 can be applied to any grammar that faces a similar problem of missing references in production rules.

XI. CONCLUSION

In this paper, we presented a multimodal retrieval system for genomics data visualizations, providing flexible visualization retrieval. To explore ways to support effective visualization retrieval, we adopted and extended different ways to represent genomics data visualizations, such as using a context-free grammar (CFG) and a state-of-the-art multimodal foundational model (BiomedCLIP). We collected 3,200 interactive genomics visualizations—spanning over 50 categories and covering a wide range of applications and design variations—and built a prototype user interface that allows users to search for genomics data visualizations using three modalities. We performed evaluation using top-k retrieval accuracy, and our notable observation is that, while specification did not translate

well as rich text in ML models, the CFG approach applied to JSON specifications showed promising retrieval results and is not computationally heavy. Future work will focus on incorporating user feedback and optimizing retrieval accuracy. First, integrating agentic AI could provide real-time support to help users understand the system and guide search results to better match user intent. Another direction is the development of a conversion system capable of translating text or images directly into Gosling specifications [57]. Finally, we plan to investigate whether projecting all three modalities: text, image, and specification, onto a shared representation space can further improve retrieval accuracy.

ACKNOWLEDGMENTS

The authors wish to thank Etowah Adams and Viet Lai for their support. With gratitude, the first author wishes to thank Professor Susan Mengel for her insightful teaching on Information Retrieval. We are also grateful for helpful feedback from anonymous reviewers. This work was supported in part by the National Institutes of Health (R01HG011773, K99HG013348, and UM1HG011536).

REFERENCES

- [1] S. Nusrat, T. Harbig, and N. Gehlenborg, "Tasks, Techniques, and Tools for Genomic Data Visualization," *Computer Graphics Forum*, vol. 38, no. 3, pp. 781–805, Jun. 2019. [Online]. Available: <https://onlinelibrary.wiley.com/doi/10.1111/cgf.13727>
- [2] S. Gundersen, S. Boddu, S. Capella-Gutierrez, F. Drablos, J. M. Fernández, R. Kompova, K. Taylor, D. Titov, D. Zerbino, and E. Hovig, "Recommendations for the fairification of genomic track metadata," *FI1000Research*, vol. 10, pp. ELIXIR-268, 2021.
- [3] A. van den Brandt, S. L'Yi, H. N. Nguyen, A. Vilanova, and N. Gehlenborg, "Understanding visualization authoring techniques for genomics data in the context of personas and tasks," *IEEE Transactions on Visualization and Computer Graphics*, 2024.
- [4] H. K. Bako, X. Liu, L. Battle, and Z. Liu, "Understanding how designers find and use data visualization examples," *IEEE Transactions on Visualization and Computer Graphics*, vol. 29, no. 1, pp. 1048–1058, 2022.
- [5] A. Satyanarayan, D. Moritz, K. Wongsuphasawat, and J. Heer, "Vega-Lite: A grammar of interactive graphics," *IEEE Trans. Vis. Comput. Graph.*, vol. 23, no. 1, pp. 341–350, Jan. 2017.
- [6] E. Hoque and M. Agrawala, "Searching the visual style and structure of d3 visualizations," *IEEE transactions on visualization and computer graphics*, vol. 26, no. 1, pp. 1236–1245, 2019.
- [7] S. Xiao, Y. Hou, C. Jin, and W. Zeng, "Wytiiwyr: A user intent-aware framework with multi-modal inputs for visualization retrieval," in *Computer Graphics Forum*, vol. 42, no. 3. Wiley Online Library, 2023, pp. 311–322.
- [8] Y. Ye, R. Huang, and W. Zeng, "Visatlas: An image-based exploration and query system for large visualization collections via neural image embedding," *IEEE Transactions on Visualization and Computer Graphics*, 2022.
- [9] Q. Chen, Y. Chen, R. Zou, W. Shuai, Y. Guo, J. Wang, and N. Cao, "Chart2vec: A universal embedding of context-aware visualizations," *IEEE Transactions on Visualization and Computer Graphics*, 2024.
- [10] M. Oppermann, R. Kincaid, and T. Munzner, "Vizcommender: Computing text-based similarity in visualization repositories for content-based recommendations," *IEEE Transactions on Visualization and Computer Graphics*, vol. 27, no. 2, pp. 495–505, 2020.
- [11] S. L'Yi, Q. Wang, F. Lekschas, and N. Gehlenborg, "Gosling: A Grammar-based Toolkit for Scalable and Interactive Genomics Data Visualization," *IEEE Transactions on Visualization and Computer Graphics*, vol. 28, no. 1, pp. 140–150, Jan. 2022. [Online]. Available: <https://ieeexplore.ieee.org/document/9557192/>
- [12] S. L'Yi and N. Gehlenborg, "Multi-view design patterns and responsive visualization for genomics data," *IEEE transactions on visualization and computer graphics*, vol. 29, no. 1, pp. 559–569, 2022.
- [13] S. L'Yi, Q. Wang, and N. Gehlenborg, "The role of visualization in genomics data analysis workflows: The interviews," in *2023 IEEE Visualization and Visual Analytics (VIS)*. IEEE, 2023, pp. 101–105.
- [14] S. L'Yi, Q. Wang, F. Lekschas, and N. Gehlenborg, "Gosling: A grammar-based toolkit for scalable and interactive genomics data visualization," *IEEE Transactions on Visualization and Computer Graphics*, vol. 28, no. 1, pp. 140–150, 2021.
- [15] J. T. Robinson, H. Thorvaldsdottir, D. Turner, and J. P. Mesirov, "igv.js: an embeddable javascript implementation of the integrative genomics viewer (igv)," *Bioinformatics*, vol. 39, no. 1, p. btac830, 2023.
- [16] C. Diesh, G. J. Stevens, P. Xie, T. De Jesus Martinez, E. A. Hershsberg, A. Leung, E. Guo, S. Dider, J. Zhang, C. Bridge *et al.*, "Jbrowse 2: a modular genome browser with views of syntenic and structural variation," *Genome biology*, vol. 24, no. 1, p. 74, 2023.
- [17] T. Manzi, S. L'Yi, and N. Gehlenborg, "Gos: a declarative library for interactive genomics visualization in python," *Bioinformatics*, vol. 39, no. 1, Jan. 2023.
- [18] S. L'Yi, A. van den Brandt, E. Adams, H. N. Nguyen, and N. Gehlenborg, "Learnable and expressive visualization authoring through blended interfaces," *IEEE Transactions on Visualization and Computer Graphics*, 2024.
- [19] T. C. Smits, S. L'Yi, A. P. Mar, and N. Gehlenborg, "Altgosling: automatic generation of text descriptions for accessible genomics data visualization," *Bioinformatics*, vol. 40, no. 12, p. btac670, 11 2024. [Online]. Available: <https://doi.org/10.1093/bioinformatics/btac670>
- [20] H. Li, Y. Wang, A. Wu, H. Wei, and H. Qu, "Structure-aware visualization retrieval," in *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*, 2022, pp. 1–14.
- [21] V. Setlur, A. Kanyuka, and A. Srinivasan, "Olio: A semantic search interface for data repositories," in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023, pp. 1–16.
- [22] G. Li and X. Yuan, "Gotreescape: Navigate and explore the tree visualization design space," *IEEE Transactions on Visualization and Computer Graphics*, vol. 29, no. 12, pp. 5451–5467, 2022.
- [23] M. J. Kusner, B. Paige, and J. M. Hernández-Lobato, "Grammar variational autoencoder," in *Proceedings of the 34th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, D. Precup and Y. W. Teh, Eds., vol. 70. PMLR, 2017, pp. 1945–1954.
- [24] J. Zhao, M. Fan, and M. Feng, "Chartseer: Interactive steering exploratory visual analysis with machine intelligence," *IEEE Transactions on Visualization and Computer Graphics*, vol. 28, no. 3, pp. 1500–1513, 2020.
- [25] G. Li, M. Tian, Q. Xu, M. J. McGuffin, and X. Yuan, "Gotree: A grammar of tree visualizations," in *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*, 2020, pp. 1–13.
- [26] Y. Kim, K. Wongsuphasawat, J. Hullman, and J. Heer, "Graphscape: A model for automated reasoning about visualization similarity and sequencing," in *Proceedings of the 2017 CHI conference on human factors in computing systems*, 2017, pp. 2628–2638.
- [27] A. Wu, W. Tong, H. Li, D. Moritz, Y. Wang, and H. Qu, "Computableviz: Mathematical operators as a formalism for visualisation processing and analysis," in *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*, 2022, pp. 1–15.
- [28] M. Sun, L. Cai, W. Cui, Y. Wu, Y. Shi, and N. Cao, "Erato: Cooperative data story editing via fact interpolation," *IEEE Transactions on Visualization and Computer Graphics*, vol. 29, no. 1, pp. 983–993, 2022.
- [29] L. Ying, A. Wu, H. Li, Z. Deng, J. Lan, J. Wu, Y. Wang, H. Qu, D. Deng, and Y. Wu, "Vaid: Indexing view designs in visual analytics system," in *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*, 2024, pp. 1–15.
- [30] H. Stitz, S. Gratzl, H. Piring, T. Zichner, and M. Streit, "Knowledge-pearls: Provenance-based visualization retrieval," *IEEE Transactions on Visualization and Computer Graphics*, vol. 25, no. 1, pp. 120–130, 2018.
- [31] H. N. Nguyen and N. Gehlenborg, "Safire: Similarity framework for visualization retrieval," in *2025 IEEE Visualization and Visual Analytics (VIS)*, 2025, pp. 246–250. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/11298749>
- [32] C. Chen, H. K. Bako, P. Yu, J. Hooker, J. Joyal, S. C. Wang, S. Kim, J. Wu, A. Ding, L. Sandeep *et al.*, "Visanatomy: An svg chart corpus with fine-grained semantic labels," *arXiv preprint arXiv:2410.12268*, 2024.
- [33] C. Chen and Z. Liu, "The state of the art in creating visualization corpora for automated chart analysis," in *Computer Graphics Forum*, vol. 42, no. 3. Wiley Online Library, 2023, pp. 449–470.
- [34] T. Wang, F. Li, L. Zhu, J. Li, Z. Zhang, and H. T. Shen, "Cross-modal retrieval: A systematic review of methods and future directions," *Proceedings of the IEEE*, vol. 112, no. 11, pp. 1716–1754, 2024.

- [35] P. P. Liang, A. Zadeh, and L.-P. Morency, “Foundations & trends in multimodal machine learning: Principles, challenges, and open questions,” *ACM Comput. Surv.*, vol. 56, no. 10, Jun. 2024. [Online]. Available: <https://doi.org/10.1145/3656580>
- [36] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark *et al.*, “Learning transferable visual models from natural language supervision,” in *International conference on machine learning*. PMLR, 2021, pp. 8748–8763.
- [37] L. H. Li, M. Yatskar, D. Yin, C.-J. Hsieh, and K.-W. Chang, “Visualbert: A simple and performant baseline for vision and language,” *arXiv preprint arXiv:1908.03557*, 2019.
- [38] J. Lu, D. Batra, D. Parikh, and S. Lee, “Vilbert: Pretraining task-agnostic visiolinguistic representations for vision-and-language tasks,” *Advances in neural information processing systems*, vol. 32, 2019.
- [39] S. L’Yi, D. Maziec, V. Stevens, T. Manz, A. Veit, M. Berselli, and others, “Chromosome: interactive multiscale visualization for structural variation in human genomes,” *Nature Methods*, 2023. [Online]. Available: <https://osf.io/pyqrx/download>
- [40] Q. Wang, X. Liu, M. Q. Liang, S. L’Yi, and N. Gehlenborg, “Enabling multimodal user interactions for genomics visualization creation,” in *Proc. IEEE VIS*, 2023. [Online]. Available: <https://osf.io/5zgye/download>
- [41] OpenAI, J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altschmidt, S. Altman, and others, “Gpt-4 technical report,” 2024. [Online]. Available: <https://arxiv.org/abs/2303.08774>
- [42] A. Parnami and M. Lee, “Learning from few examples: A summary of approaches to few-shot learning,” 2022. [Online]. Available: <https://arxiv.org/abs/2203.04291>
- [43] M. J. Kusner, B. Paige, and J. M. Hernández-Lobato, “Grammar variational autoencoder,” in *International conference on machine learning*. PMLR, 2017, pp. 1945–1954.
- [44] Z. Li, X. Zhang, Y. Zhang, D. Long, P. Xie, and M. Zhang, “Towards general text embeddings with multi-stage contrastive learning,” *arXiv preprint arXiv:2308.03281*, 2023.
- [45] A. Yang, B. Yang, B. Hui, B. Zheng, B. Yu, C. Zhou, C. Li, C. Li, D. Liu, F. Huang, and others, “Qwen2 technical report,” 2024.
- [46] M. Henderson, R. Al-Rfou, B. Strophe, Y.-H. Sung, L. Lukács, R. Guo, S. Kumar, B. Miklos, and R. Kurzweil, “Efficient natural language response suggestion for smart reply,” *arXiv preprint arXiv:1705.00652*, 2017.
- [47] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net, 2019. [Online]. Available: <https://openreview.net/forum?id=Bkg6RiCqY7>
- [48] S. Zhang, Y. Xu, N. Usuyama, H. Xu, J. Bagga, R. Tinn, S. Preston, R. Rao, M. Wei, N. Valluri, C. Wong, A. Tupini, Y. Wang, M. Mazzola, S. Shukla, L. Liden, J. Gao, A. Crabtree, B. Piening, C. Bifulco, M. P. Lungren, T. Naumann, S. Wang, and H. Poon, “A multimodal biomedical foundation model trained from fifteen million image–text pairs,” *NEJM AI*, vol. 2, no. 1, p. AIoa2400640, 2025. [Online]. Available: <https://ai.nejm.org/doi/full/10.1056/AIoa2400640>
- [49] S. L’Yi, Y. Chang, D. Shin, and J. Seo, “Toward understanding representation methods in visualization recommendations through scatterplot construction tasks,” in *Computer Graphics Forum*, vol. 38, no. 3. Wiley Online Library, 2019, pp. 201–211.
- [50] Meta Platforms, Inc., “A React JavaScript library for building user interfaces,” <https://github.com/facebook/react>, accessed: 2025-3-31.
- [51] M. Grinberg, *Flask web development: developing web applications with python*. USA: O’Reilly Media, Inc., 2018.
- [52] G. Poesia, O. Polozov, V. Le, A. Tiwari, G. Soares, C. Meek, and S. Gulwani, “Synchronesh: Reliable code generation from pre-trained language models,” *arXiv preprint arXiv:2201.11227*, 2022.
- [53] C. Diesh, “Awesome genome visualization,” <https://cmdcolin.github.io/awesome-genome-visualization/>, 2024, accessed: 2025-09-09.
- [54] D. Karolchik, R. Baertsch, M. Diekhans, T. S. Furey, A. Hinrichs, Y. Lu, K. M. Roskin, M. Schwartz, C. W. Sugnet, D. J. Thomas *et al.*, “The ucsc genome browser database,” *Nucleic acids research*, vol. 31, no. 1, pp. 51–54, 2003.
- [55] M. Gonçalves, C. Cardoso, and P. Badke-Schaub, “Inspiration choices that matter: the selection of external stimuli during ideation,” *Design Science*, vol. 2, p. e10, 2016.
- [56] A. Baigelenov, P. Shukla, and P. Parsons, “How visualization designers perceive and use inspiration,” in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, ser. CHI ’25. New York, NY, USA: Association for Computing Machinery, 2025. [Online]. Available: <https://doi.org/10.1145/3706598.3714191>
- [57] Q. Wang, X. Liu, M. Q. Liang, S. L’Yi, and N. Gehlenborg, “Enabling multimodal user interactions for genomics visualization creation,” in *2023 IEEE Visualization and Visual Analytics (VIS)*. IEEE, Oct. 2023, pp. 111–115.



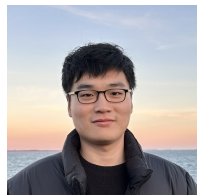
Huyen N. Nguyen is a Postdoctoral Research Fellow in Biomedical Informatics at Harvard University, having received her PhD in Computer Science from Texas Tech University. Her research focuses on developing visualization systems that enable efficient data exploration and interaction, with a special focus on visualization retrieval and multimodal applications.



Sehi L’Yi received PhD in computer science and engineering from Seoul National University, in 2020. He is currently an NIH K99 Postdoctoral Fellow in Biomedical Informatics at Harvard University. His research bridges visualization, human–computer interaction, and biomedical informatics, with the goal of accelerating data interpretation, analysis, and communication within biomedical domains through scalable and AI-powered interactive data systems.



Thomas C. Smits got his Master in Biomedical Informatics at Harvard Medical School and subsequently worked as an Associate specializing in multi-modal genomics and single cell atlas data visualizations. He is currently pursuing his PhD in computational immunology at the Radboud University Medical Center in Nijmegen, the Netherlands.



Shanghua Gao is a Research Fellow at Harvard University, following his Ph.D. at Nankai University. His research spans foundational AI models—including representation learning, generative modeling, and efficient architectures—as well as AI for Science, where he develops autonomous, agentic AI systems for biomedical reasoning and scientific discovery.



Marinka Zitnik is an Associate Professor of Biomedical Informatics at Harvard Medical School, Associate Faculty at Kempner Institute for the Study of Natural and Artificial Intelligence at Harvard University, and Associate Member at Broad Institute of MIT and Harvard. Zitnik investigates the foundations of AI for scientific discovery and medicine.



Nils Gehlenborg is an Associate Professor of Biomedical Informatics at Harvard Medical School and affiliate faculty at the Broad Institute of MIT and Harvard. His research focuses on developing interfaces and algorithms that enable humans to interact with biomedical data.